

Stacks

Stacks are a fundamental data structure in computer science, characterized by their Last In, First Out (LIFO) behavior. This means that the last element added to the stack will be the first one to be removed. Below are the essential operations and functions that can be performed on a stack:

1. **Push**

- **Description**: This function adds an element to the top of the stack.
- **Syntax**: `stack.push(element);`
- **Example**: If `stack` is of type `int`, `stack.push(5);` will place the integer `5` on the top of the stack.

2. **Pop**

- **Description**: This function removes the element from the top of the stack and returns it.
- **Syntax**: `stack.pop();`
- **Example**: If the stack's top element is `5`, calling `stack.pop();` will remove `5` and reduce the stack size.

3. **Top / Peek**

- **Description**: This function retrieves the top element of the stack without removing it.
- **Syntax**: `stack.top();` or `stack.peek();`
- **Example**: If the top element is `5`, calling `stack.top();` will return `5` without modifying the stack.

4. **IsEmpty**

- **Description**: This function checks whether the stack is empty, returning a boolean value.
- **Syntax**: `stack.isEmpty();`
- **Example**: If the stack is empty, `stack.isEmpty();` will return `true`.

5. **Size**

- **Description**: This function returns the number of elements currently in the stack.
- **Syntax**: `stack.size();`
- **Example**: If there are three elements in the stack, `stack.size();` will return `3`.

6. **Clear**

- **Description**: This function removes all elements from the stack.
- **Syntax**: `stack.clear();`
- **Example**: After calling `stack.clear();`, the stack will be empty.

7. **Copy Constructor and Assignment Operator**

- **Description**: These functions allow for the creation of a new stack that is a copy of an existing stack or the assignment of one stack to another.
- **Example**: `Stack newStack(existingStack);` will create `newStack` as a copy of `existingStack`.

Example Code Using Stack in C++

Here is a simple demonstration of stack operations in C++ using the Standard Template Library (STL):

```
```cpp
#include <iostream>
#include <stack> // Include the stack header
using namespace std;

int main() {
 stack<int> myStack; // Create a stack of integers
```

```

// Push elements onto the stack
myStack.push(10);
myStack.push(20);
myStack.push(30);

// Display the top element
cout << "Top element: " << myStack.top() << endl; // Outputs: 30

// Pop the top element
myStack.pop();
cout << "Top element after pop: " << myStack.top() << endl; // Outputs: 20

// Check the size of the stack
cout << "Current stack size: " << myStack.size() << endl; // Outputs: 2

// Check if the stack is empty
cout << "Is stack empty? " << (myStack.empty() ? "Yes" : "No") << endl; // Outputs: No

// Clear the stack
while (!myStack.empty()) {
 myStack.pop(); // Remove all elements
}

cout << "Stack size after clearing: " << myStack.size() << endl; // Outputs: 0
return 0;
}
...

```

### ### Summary

The above operations define a robust interface for managing data in a stack structure, facilitating various applications such as expression evaluation, backtracking algorithms, and memory management. Understanding and utilizing these functions effectively is essential for implementing stacks in software development and algorithm design. If further clarification or examples are required, please feel free to inquire.